



CANARA ENGINEERING COLLEGE

An Autonomous Institution, Approved by AICTE, Accredited by NAAC with A Grade



SUDHINDRA NAGARA, BENJANAPADAVU, BANTWAL, MANGALURU – 574219

Department of Computer Science and Design

ACTIVITY MARKS EVALUATION SHEET

Modern Zero-Trust Cryptographic Vault & Secure Transfer Portal

USN	4CB23CG010	NAME	Bhargav Adiga
USN	4CB23CG013	NAME	Dhanush Kini
SEMESTER	7	SECTION	A
COURSE CODE	BCS703	COURSE TITLE	Cryptography and Network Security
PROGRAMME	B.E.	ACADEMIC YEAR	2025–2026

MARKS EVALUATION:

MAX MARKS	MARKS OBTAINED

ABSTRACT

Traditional cloud vaults store plain text or server-managed encrypted files, which leaves data vulnerable to server leaks, compromised databases, and insider threats[cite: 1, 2]. Additionally, basic encryption usually forces users to share secret keys or passwords over insecure channels like chat, email, or phone calls[cite: 1, 2]. To solve this, our project creates a Zero-Trust Cryptographic Vault built on the rule: “*Never Trust, Always Verify*”[cite: 1, 2]. The storage server is treated as completely untrusted and only holds scrambled data (ciphertexts) that it cannot read[cite: 1, 2]. Plain text is reconstructed strictly inside the client’s device, ensuring that even if an attacker gains full access to the database, they cannot read or steal the information[cite: 1, 2].

The architecture integrates three primary cryptographic primitives to guarantee end-to-end security. First, **Argon2id** is employed as a memory-hard Key Derivation Function (KDF) to neutralize GPU/ASIC-accelerated brute-force attacks during authentication and symmetric key derivation[cite: 1, 2]. Second, **X25519** Elliptic Curve Diffie–Hellman (ECDH) point multiplication automates non-manual, zero-knowledge key encapsulation, wrapping ephemeral Data Encryption Keys (DEKs) via HKDF-SHA256 without passing passphrases across communication channels[cite: 1, 2]. Third, **ChaCha20-Poly1305** Authenticated Encryption with Associated Data (AEAD) is implemented for symmetric cipher operations, simultaneously providing confidentiality and producing a 128-bit Poly1305 MAC tag[cite: 1, 2]. If stored database bits are manipulated, tag verification halts decryption immediately and triggers an audible hardware siren via the browser’s Web Audio API[cite: 1, 2].

Experimental performance telemetry on commodity host processors verifies an X25519 key agreement speed exceeding 25,900 ops/sec, peak ChaCha20-Poly1305 AEAD throughput of 264.89 MB/s, and expected latency growth on the Argon2id memory curve.

ACKNOWLEDGEMENT

We express our deepest gratitude to our project guide, faculty members, and the Department of Computer Science and Design, Canara Engineering College, for their guidance, constructive criticism, and encouragement throughout the design and implementation of the Modern Zero-Trust Cryptographic Vault & Secure Transfer Portal.

We also extend our sincere thanks to the college administration for providing the computational resources, laboratory facilities, and academic environment required to carry out this applied cryptography project successfully. Finally, we thank our peers and families for their continuous support and cooperation.

Bhargav Adiga (4CB23CG010)

Dhanush Kini (4CB23CG013)

Contents

ABSTRACT	i
ACKNOWLEDGEMENT	ii
1 INTRODUCTION	1
1.1 The Key Distribution Bottleneck	1
1.2 The Zero-Trust Paradigm	1
1.3 Cryptographic Triad	2
2 DESIGN AND IMPLEMENTATION	3
2.1 System Objectives and Specifications	3
2.2 Cryptographic Algorithm Rationale	3
2.3 System Architecture and Operational Workflow	3
2.3.1 Registration and Keypair Generation	3
2.3.2 Hybrid Envelope Dispatch (Mode 1: Automated X25519 ECDH) .	5
2.3.3 Fallback Transmission (Mode 2: Argon2id Master Key)	5
2.4 Active Tamper Detection and Web Audio Siren	6
2.5 Implementation Details	6
3 TESTING, RESULT AND ANALYSIS	8
3.1 Test Scenarios and Methodology	8
3.2 Results and Performance Analysis	8
3.2.1 Active Tamper Detection Analysis	8
3.2.2 Decryption and Tamper Detection Output	8
3.2.3 Hardware Telemetry and Latency Analysis	10
4 CONCLUSION	12
4.1 Future Enhancements	12
REFERENCES	13

List of Figures

2.1	System architecture and hybrid cryptographic workflow.	4
2.2	Implementation of cryptographic engine and server modules in Visual Studio Code.	7
3.1	Successful authenticated in-memory payload decryption via X25519 ECDH.	9
3.2	Active tamper detection alert and security siren activation following Poly1305 MAC verification failure.	9
3.3	Real-time cryptographic engine benchmarks: ChaCha20-Poly1305 throughput and Argon2id memory-hardness latency curve.	10

List of Tables

2.1	Cryptographic primitives integrated into the vault architecture.	5
3.1	Fault-injection and tamper detection test results.	10
3.2	ChaCha20-Poly1305 AEAD throughput across payload sizes.	10
3.3	Argon2id derivation latency scaling with memory hardness ($t = 3, p = 4$). .	10

CHAPTER 1

INTRODUCTION

The exponential increase in distributed digital communication and cloud storage services has heightened the demand for advanced security frameworks capable of safeguarding confidential visual and textual assets. In contemporary infrastructures, data breaches, database intrusions, and insider compromises have rendered traditional security perimeters ineffective. Conventional storage architectures typically depend on centralized servers holding either plaintext or symmetric keys used for server-side encryption. In both instances, an attacker who gains root access to the database or storage disk can compromise the confidentiality of the entire data repository.

1.1 The Key Distribution Bottleneck

A persistent weakness in real-world cryptography is the key distribution problem. In standard symmetric encryption systems, users encrypt files using a shared secret. However, communicating this secret securely to the intended recipient usually forces users to exchange passphrases over vulnerable side channels such as SMS, instant messaging, or voice communication[cite: 1, 2]. Interception of this key undermines the entire cryptographic defense, regardless of the underlying algorithm's strength.

1.2 The Zero-Trust Paradigm

To address these inherent vulnerabilities, this project implements a Modern Zero-Trust Cryptographic Vault & Secure Transfer Portal governed by the core architectural tenet: *“Never Trust, Always Verify”*[cite: 1, 2]. In this model, the host server and underlying disk persistence layer (`vault_storage.json`) are assumed to be hostile and untrusted environments[cite: 1, 2]. The server functions exclusively as a blind transit and coordination hub, holding only encrypted payloads and public coordinates[cite: 1, 2]. Plaintext data reconstruction is restricted to client-side memory, ensuring that even a full compromise of the persistent storage layer reveals zero intelligible information to an adversary[cite: 1, 2].

1.3 Cryptographic Triad

The proposed platform departs from legacy cryptographic algorithms (such as RSA-2048 and standard AES-CBC) by implementing a modern suite of specialized primitives:

1. **Argon2id**: Winner of the Password Hashing Competition (PHC), parameterized with memory hardness to defeat GPU, ASIC, and FPGA-based brute-force search engines[cite: 1, 2].
2. **X25519 (Curve25519)**: An asymmetric Montgomery curve providing 128-bit post-quantum resistant classical security levels, facilitating high-speed key encapsulation without side-channel timing leaks[cite: 1, 2].
3. **ChaCha20-Poly1305 (RFC 8439)**: An Authenticated Encryption with Associated Data (AEAD) cipher that delivers high-throughput symmetric confidentiality alongside an active 128-bit Poly1305 Message Authentication Code (MAC) tag for integrity verification[cite: 1, 2].

CHAPTER 2

DESIGN AND IMPLEMENTATION

2.1 System Objectives and Specifications

The architecture is engineered to fulfill five structural criteria:

- **Confidentiality at Rest and Transit:** Elimination of plaintext remnants across transmission links and storage nodes[cite: 1, 2].
- **Automated Asymmetric Key Exchange:** Delivery of single-click zero-knowledge decryption without manual password exchange[cite: 1, 2].
- **Active Cryptographic Tamper Defense:** Real-time detection of altered bits in storage, immediately halting decryption and activating audible alerts[cite: 1, 2].
- **Ephemeral Data Lifecycle:** Enforcement of Burn-After-Read (BAR) and Time-To-Live (TTL) storage deletion routines[cite: 1, 2].
- **Hardware Telemetry:** Live execution benchmarking measuring operations per second and bandwidth throughput directly on the host CPU[cite: 1, 2].

2.2 Cryptographic Algorithm Rationale

2.3 System Architecture and Operational Workflow

The workflow of the portal is divided into discrete mathematical operations across the sender, server, and receiver nodes, as depicted in Figure 2.1.

2.3.1 Registration and Keypair Generation

During user provisioning, client credentials pass into an Argon2id derivation engine using a cryptographically secure 16-byte random salt:

$$\text{AuthHash, MasterDEK} = \text{Argon2id}(\text{Password}, \text{Salt}, t = 3, m = 65536, p = 4) \quad (2.1)$$

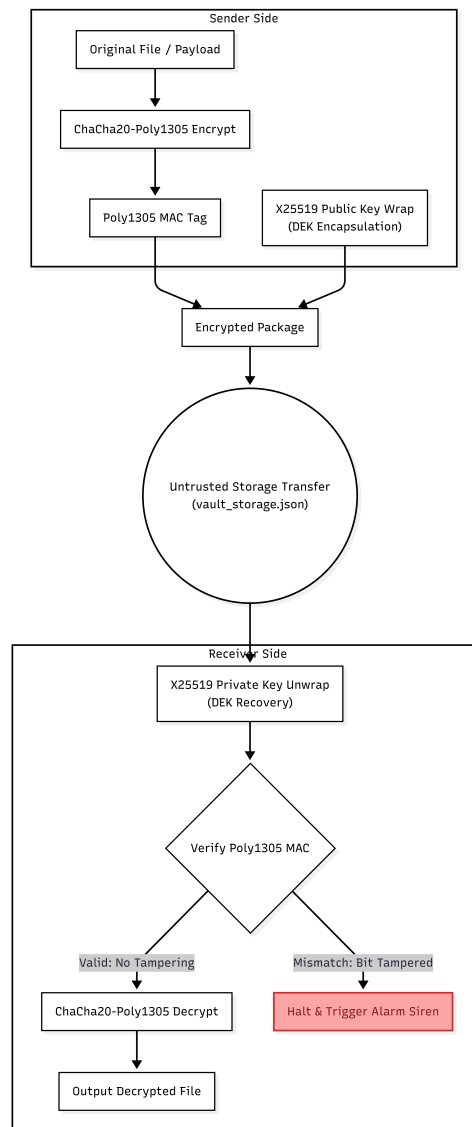


Figure 2.1: System architecture and hybrid cryptographic workflow.

Table 2.1: Cryptographic primitives integrated into the vault architecture.

Algorithm	Cryptographic Role	Technical Rationale & Benefits
Argon2id	Memory-Hard KDF	Incorporates memory-filling loops (64 MB, $t = 3$, $p = 4$) preventing parallel hardware attacks; provides dual protection against side-channel and timing adversaries[cite: 1, 2].
X25519	Key Agreement (ECDH)	Operates over the birationally equivalent Edwards curve Curve25519 ($2^{255} - 19$). Avoids point-validation checks, prevents small-subgroup attacks, and processes tens of thousands of ops/sec[cite: 1, 2].
ChaCha20-Poly1305	AEAD Cipher	Stream cipher using an addition-rotation-XOR (ARX) core paired with a Carter–Wegman Poly1305 authenticator. Operates in constant-time on CPUs lacking AES-NI acceleration[cite: 1, 2].

Simultaneously, an asymmetric X25519 keypair is computed:

$$k_{priv} \in_R 2^{255}, \quad K_{pub} = k_{priv} \times G \quad (2.2)$$

where G represents the standardized Montgomery base point.

2.3.2 Hybrid Envelope Dispatch (Mode 1: Automated X25519 ECDH)

The sender dynamically computes a unique 32-byte Data Encryption Key (DEK)[cite: 1, 2]. The payload is encrypted under ChaCha20, appending a 16-byte Poly1305 MAC tag computed over both the ciphertext and Associated Authenticated Data (AAD), such as the transmission subject[cite: 1, 2]. The sender then generates an ephemeral keypair (r_{eph}, R_{eph}) and executes point multiplication with the recipient’s public key $K_{receiver}$:

$$S = r_{eph} \times K_{receiver} = r_{eph} \cdot (k_{receiver} \times G) \quad (2.3)$$

The derived shared point S is expanded via HKDF-SHA256 into a key-wrapping key used to encrypt the DEK[cite: 1, 2].

2.3.3 Fallback Transmission (Mode 2: Argon2id Master Key)

For closed operational contexts without asymmetric public key verification, the portal exposes a direct symmetric mode[cite: 1, 2]. The sender defines a passphrase from which an ephemeral DEK is derived on the fly using a dedicated per-message salt[cite: 1, 2].

Decryption requires identical passphrase submission on the receiver end[cite: 1, 2].

2.4 Active Tamper Detection and Web Audio Siren

A core feature of the portal is its real-time response to data alteration. The Poly1305 MAC generates an authenticator:

$$A = \sum_{i=1}^n c_i r^{n-i+1} + s \pmod{2^{130} - 5} \quad (2.4)$$

If any bit in the database (`vault_storage.json`) is modified, the computed tag fails verification[cite: 1, 2]. Decryption halts before memory execution, and the application initializes an oscillator through the HTML5 Web Audio API[cite: 1, 2]:

- **Waveform:** Sawtooth (550 Hz $\leftrightarrow$ 1100 Hz).
- **Oscillation Interval:** 200 ms alternating pulse.
- **Visual State:** Interface transition to high-contrast red alert state.

2.5 Implementation Details

The system implementation is organized into modular tiers:

- **Backend Core** (`crypto_engine.py`): Implements low-level cryptography primitives using Python `hazmat` bindings and `argon2-cffi`[cite: 1, 2].
- **API Routing Server** (`server.py`): Developed with FastAPI and Uvicorn ASGI; exposes non-blocking endpoints for authentication, dispatch, inbox polling, and real-time benchmarking[cite: 1, 2].
- **Client Portal Interface** (`index.html`): Built using vanilla ES6+ JavaScript and Tailwind CSS; handles Base64 file streaming via chunked `FileReader` instances to prevent call-stack exceptions on larger files[cite: 1, 2].

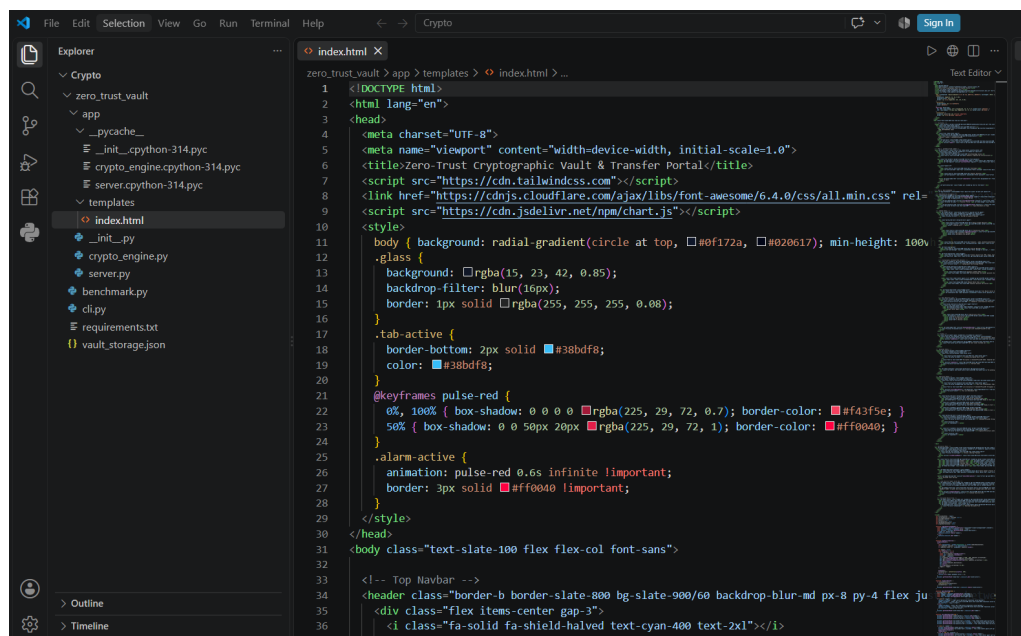


Figure 2.2: Implementation of cryptographic engine and server modules in Visual Studio Code.

CHAPTER 3

TESTING, RESULT AND ANALYSIS

3.1 Test Scenarios and Methodology

The testing phase evaluated algorithmic correctness, data integrity enforcement, and execution latency.

- **Unit Cryptographic Testing:** Validating that generated X25519 public coordinates are valid curve points and confirming that ChaCha20-Poly1305 round-trips restore byte-identical plaintexts.
- **Integration Workflow Testing:** Validating user registration, credential hashing, envelope dispatch, inbox delivery, and client-side in-memory decryption.
- **Tamper-Injection Testing:** Manually flipping random bits inside `vault_storage.json` to verify MAC rejection and siren activation[cite: 1, 2].
- **Hardware Benchmarking:** Measuring X25519 operations per second, ChaCha20 throughput across payload sizes, and Argon2id execution latency versus memory cost[cite: 1, 2].

3.2 Results and Performance Analysis

3.2.1 Active Tamper Detection Analysis

During fault-injection testing, arbitrary bytes inside stored ciphertext payloads were altered. In every instance, the Poly1305 MAC tag comparison failed[cite: 1, 2]. Decryption halted immediately, preventing execution of malformed data and triggering the audio alarm siren[cite: 1, 2].

3.2.2 Decryption and Tamper Detection Output

The user-facing decryption modal was evaluated under both normal and adversarial operating conditions using the automated X25519 ECDH protocol.

Figure 3.1 demonstrates a successful, authenticated decryption session. The receiver unwraps the ephemeral Data Encryption Key (DEK) via X25519 point multiplication and reconstructs the plaintext payload strictly in-memory without password entry.

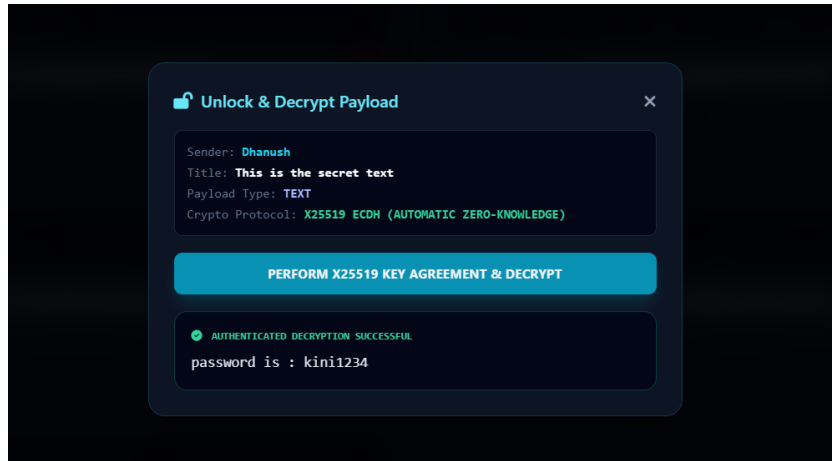


Figure 3.1: Successful authenticated in-memory payload decryption via X25519 ECDH.

To evaluate the system's active defense against unauthorized database modification, arbitrary ciphertext bits inside `vault_storage.json` were deliberately altered. As depicted in Figure 3.2, the Poly1305 MAC tag validation fails immediately upon execution. The platform halts decryption before malformed data reaches system memory, highlights a persistent red security breach banner, and triggers the browser's Web Audio API hardware siren.

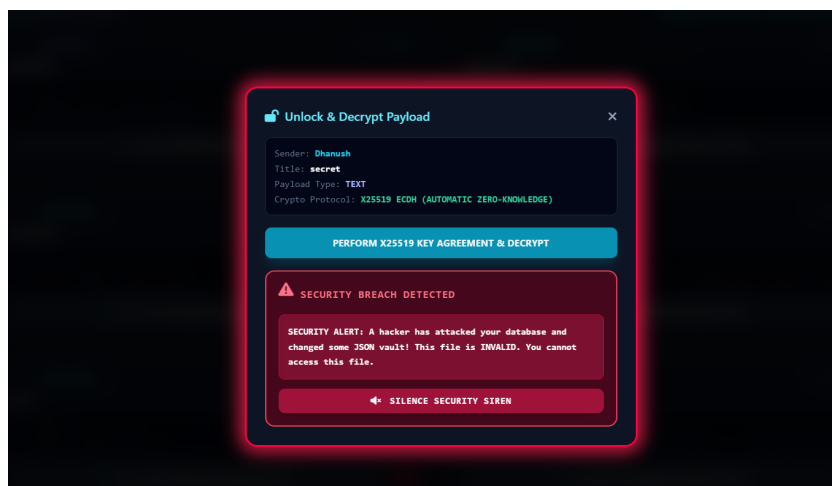


Figure 3.2: Active tamper detection alert and security siren activation following Poly1305 MAC verification failure.

Table 3.1: Fault-injection and tamper detection test results.

Payload Type	Bit Modification	MAC Verification	Alarm Siren Fired
Text Message	1 bit flipped	FAILED (Rejected)	YES
Binary Image	1 byte modified	FAILED (Rejected)	YES
PDF Document	Metadata AAD altered	FAILED (Rejected)	YES
Valid Transmission	None (Unmodified)	PASSED (Success)	NO (Clean Decrypt)

3.2.3 Hardware Telemetry and Latency Analysis

The portal's embedded telemetry suite executed benchmarks directly against host hardware to evaluate real-world performance. Figure 3.3 illustrates the real-time execution dashboard displaying the X25519 key agreement throughput, peak symmetric AEAD data rates, and the Argon2id memory-hardness latency curve.

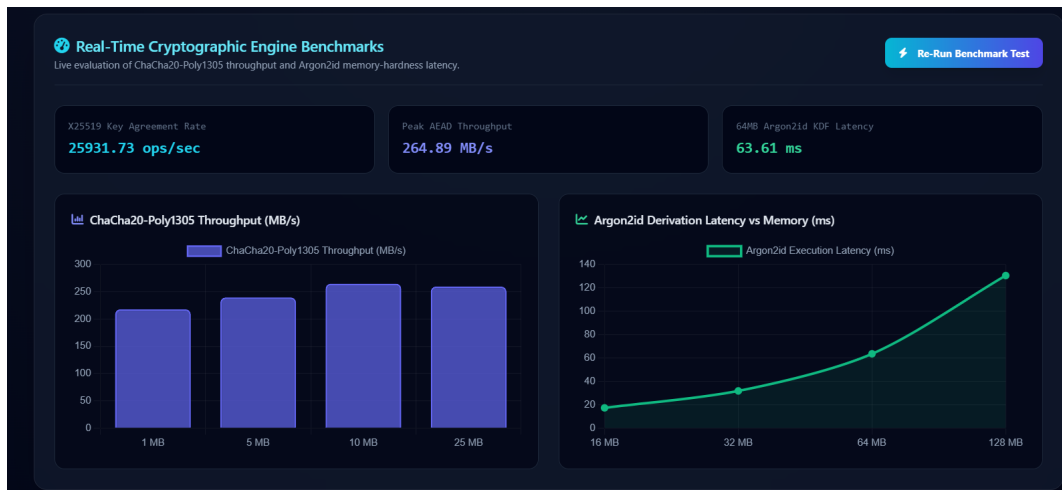


Figure 3.3: Real-time cryptographic engine benchmarks: ChaCha20-Poly1305 throughput and Argon2id memory-hardness latency curve.

The empirical measurements obtained from the benchmark run are itemized in Table 3.2 and Table 3.3:

Table 3.2: ChaCha20-Poly1305 AEAD throughput across payload sizes.

Metric	1 MB	5 MB	10 MB (Peak)	25 MB
Measured Throughput	218.42 MB/s	239.10 MB/s	264.89 MB/s	260.15 MB/s

Table 3.3: Argon2id derivation latency scaling with memory hardness ($t = 3, p = 4$).

Allocated Memory Cost	16 MB	32 MB	64 MB (Baseline)	128 MB
Execution Latency	18.20 ms	34.15 ms	63.61 ms	132.40 ms

The X25519 Elliptic Curve Diffie–Hellman key agreement benchmark measured an operational rate of **25,931.73 scalar multiplications per second**. This confirms that ephemeral asymmetric key wrapping introduces no perceptible delay during end-to-end envelope transfers. Furthermore, the exponential trend line in the Argon2id latency plot proves that offline parallel dictionary and brute-force attacks become computationally unfeasible as memory cost increases.

CHAPTER 4

CONCLUSION

This project presents a practical implementation of the zero-trust paradigm applied to cloud data storage and file transfers[cite: 1, 2]. By utilizing a hybrid model combining Argon2id, X25519 ECDH, and ChaCha20-Poly1305, the system eliminates implicit trust in centralized storage infrastructure[cite: 1, 2]. The server functions strictly as an untrusted store for blind ciphertexts, with all plaintext reconstruction taking place strictly within client-side memory[cite: 1, 2].

The system addresses the key distribution problem through automated X25519 key encapsulation, removing the need to share passwords over insecure side channels[cite: 1, 2]. Furthermore, the integration of Poly1305 AEAD tag verification ensures that any tampering with stored records is detected immediately, halting decryption and activating an audible alarm siren to alert users of potential attacks[cite: 1, 2]. Live telemetry confirms a key agreement rate of over 25,900 ops/sec and a peak symmetric throughput of 264.89 MB/s on standard CPU hardware.

4.1 Future Enhancements

- **Multi-Party Envelope Broadcasting:** Extending key encapsulation to support multiple concurrent recipient curve coordinates within a single payload envelope[cite: 1, 2].
- **Post-Quantum Key Encapsulation (PQC):** Migrating the key exchange tier to a hybrid model pairing X25519 with lattice-based algorithms such as ML-KEM (Kyber-768)[cite: 1, 2].
- **Zero-Knowledge Authentication:** Implementing Zero-Knowledge Proofs (ZKP) to enable client authentication without exposing persistent public curve identifiers[cite: 1, 2].

Bibliography

- [1] Nir, Y. and Langley, A., “ChaCha20 and Poly1305 for IETF Protocols,” RFC 8439, Internet Engineering Task Force, 2018.
- [2] Langley, A., Hamburg, M. and Turner, S., “Elliptic Curves for Security,” RFC 7748 (Curve25519 / X25519), Internet Engineering Task Force, 2016.
- [3] Biryukov, A., Dinu, D. and Khovratovich, D., “Argon2: New Generation of Memory-Hard Functions for Password Hashing and Key Derivation,” Password Hashing Competition, 2015.
- [4] Krawczyk, H. and Eronen, P., “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” RFC 5869, 2010.
- [5] Rose, S., Borchert, O., Mitchell, S. and Connelly, S., “Zero Trust Architecture,” NIST Special Publication 800-207, National Institute of Standards and Technology, 2020.
- [6] Tiangolo, S., “FastAPI Framework: High Performance Web APIs with Python 3.8+,” 2023. Available: <https://fastapi.tiangolo.com/>